

What is HTTP — Revision Notes

Key Concepts

Term	Simple Definition
HTTP	HyperText Transfer Protocol — rules for sending data between browser and server
HTTPS	HTTP + TLS encryption — data is encrypted in transit
Stateless	Each request is independent — server remembers nothing between requests
Request	Message from client to server asking for something
Response	Server's reply to the request
Header	Metadata attached to request/response (content type, auth token, etc.)
Body	Actual data payload (JSON, HTML, file, etc.)
TLS	Transport Layer Security — encryption layer under HTTPS

HTTP Methods

Method	Purpose	Has Body?	Idempotent?
GET	Fetch data	No	Yes
POST	Create new resource	Yes	No
PUT	Replace entire resource	Yes	Yes
PATCH	Update part of resource	Yes	No
DELETE	Remove resource	No	Yes
HEAD	Like GET but returns only headers	No	Yes
OPTIONS	Ask server what methods are allowed	No	Yes

Idempotent = calling it multiple times gives same result (safe to retry)

Status Codes

Range	Meaning	Key Examples
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 302 Found, 304 Not Modified

Range	Meaning	Key Examples
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

HTTP Request Structure

```
GET /api/users/123 HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGc...
Content-Type: application/json
```

HTTP Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=3600

{ "id": 123, "name": "Anil" }
```

HTTP vs HTTPS

	HTTP	HTTPS
Encryption	None	TLS encrypted
Port	80	443
Data visible to network?	Yes	No
Use in production?	Never	Always

HTTP Versions

Version	Key Feature
HTTP/1.0	New TCP connection per request
HTTP/1.1	Persistent connections, keep-alive
HTTP/2	Multiplexing, header compression, single TCP connection
HTTP/3	QUIC (UDP-based), no head-of-line blocking

Industry Examples

- **GitHub API** — uses 422 Unprocessable Entity for validation errors (not 400)
 - **Stripe** — idempotency keys on POST requests to prevent duplicate charges on retry
 - **Twitter/X** — uses 429 Too Many Requests for rate limiting API consumers
 - **Google** — forced migration of all sites to HTTPS; Chrome marks HTTP sites as "Not Secure"
-

Interview Questions

Q1: What is the difference between PUT and PATCH?

PUT replaces the entire resource. PATCH updates only specified fields. If you PUT a user object missing the email field, email gets wiped. PATCH only changes what you send.

Q2: What does stateless mean in HTTP?

Each HTTP request is completely independent. The server has no memory of previous requests. This is why we need tokens/cookies — to carry identity on each request.

Q3: What is the difference between 401 and 403?

401 = not authenticated (who are you?). 403 = authenticated but not authorized (I know who you are, but you can't do this).

Q4: Why is idempotency important in APIs?

Networks fail. Clients retry. If POST /payment is not idempotent, a retry could charge the user twice. Idempotency keys (like Stripe uses) ensure retries don't duplicate side effects.

Q5: What happens in HTTPS that doesn't happen in HTTP?

Before any data is sent, a TLS handshake occurs — server presents a certificate, client verifies it, they agree on an encryption key. All subsequent data is encrypted. An attacker intercepting packets sees only gibberish.